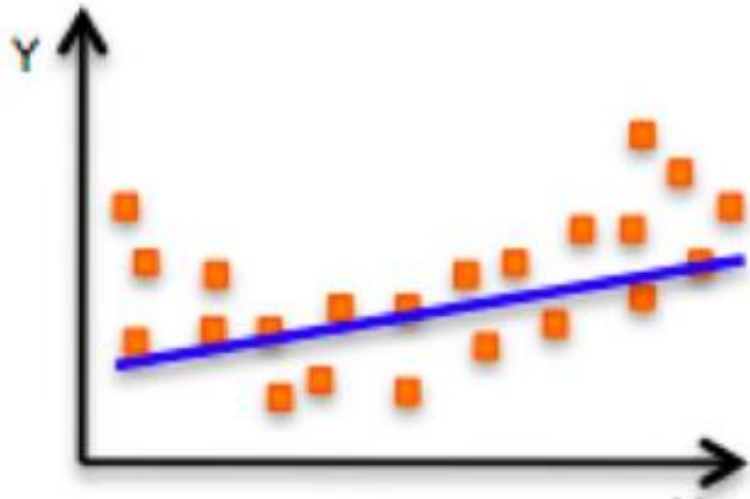


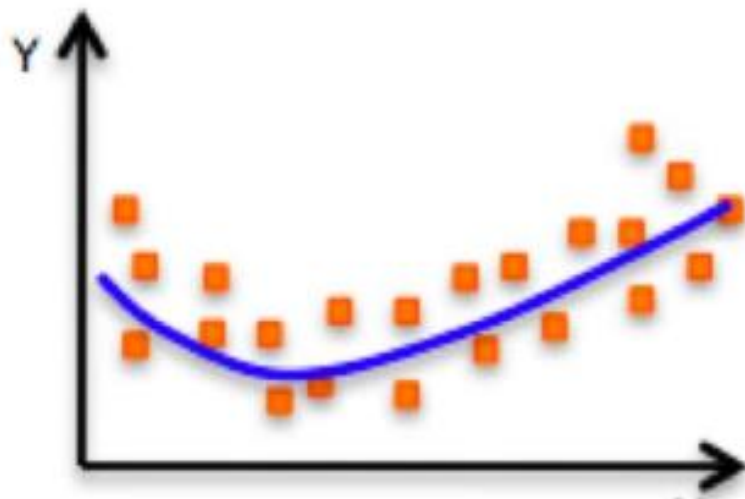
Practical tricks: Data Distribution, Overfitting,  
Regularization, Hyper-Parameters Tuning,  
Normalization, vanishing & exploding gradients

# The Problem of Overfitting

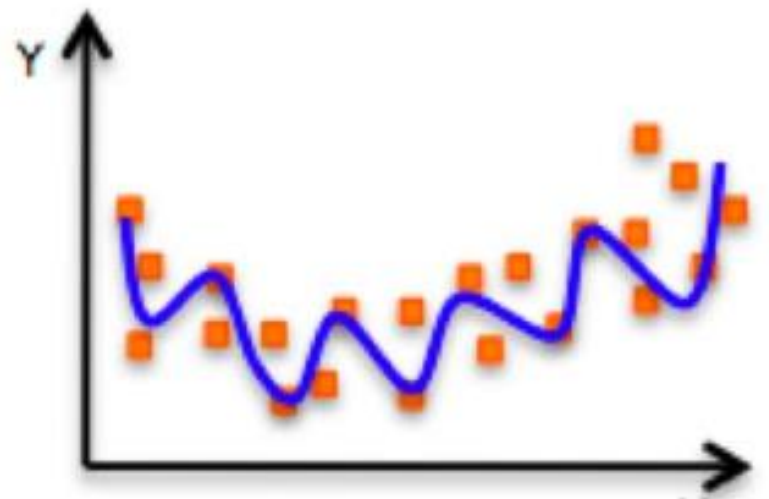


## Underfitting

Model does not have capacity to fully learn the data



## Ideal fit

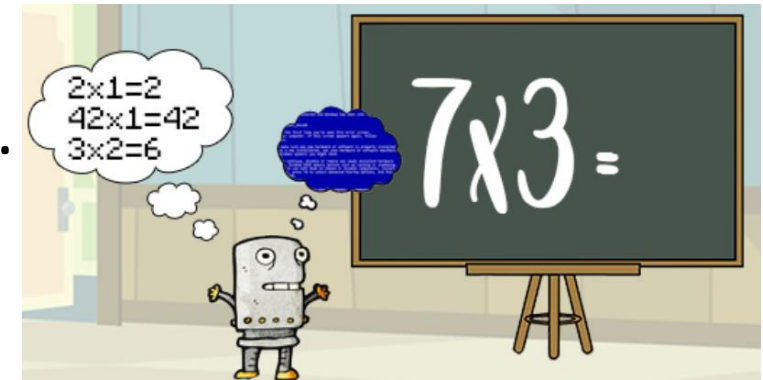


## Overfitting

Too complex, extra parameters, does not generalize well

# Overfitting

- ❑ Memorizing is not learning!
- ❑ What is overfitting?
  - The word **overfitting** refers to a model that models the training data too well. Instead of learning the general **distribution** of the data, the model learns the *expected output* for every data point.
- ❑ This is the same as **memorizing the answers** to a math's quiz instead of **knowing the formulas**. Because of this, the model cannot *generalize*.
- ❑ Everything is all good as long as you are in *familiar territory*, but as soon as you step outside, you're lost.



# How to detect Overfitting?

- ❑ To test this ability, a simple method consists in splitting the dataset into two parts: the **training set** and the **test set**.

|                    |                        |                                                                          |
|--------------------|------------------------|--------------------------------------------------------------------------|
|                    | Low Training Error     | High Training Error                                                      |
| Low Testing Error  | The model is learning! | Probably some error in your code. Or you've created a <i>psychic</i> AI. |
| High Testing Error | OVERFITTING            | The model is not learning.                                               |

Overfitting can be seen as the **difference** between the **training** and **testing** error.

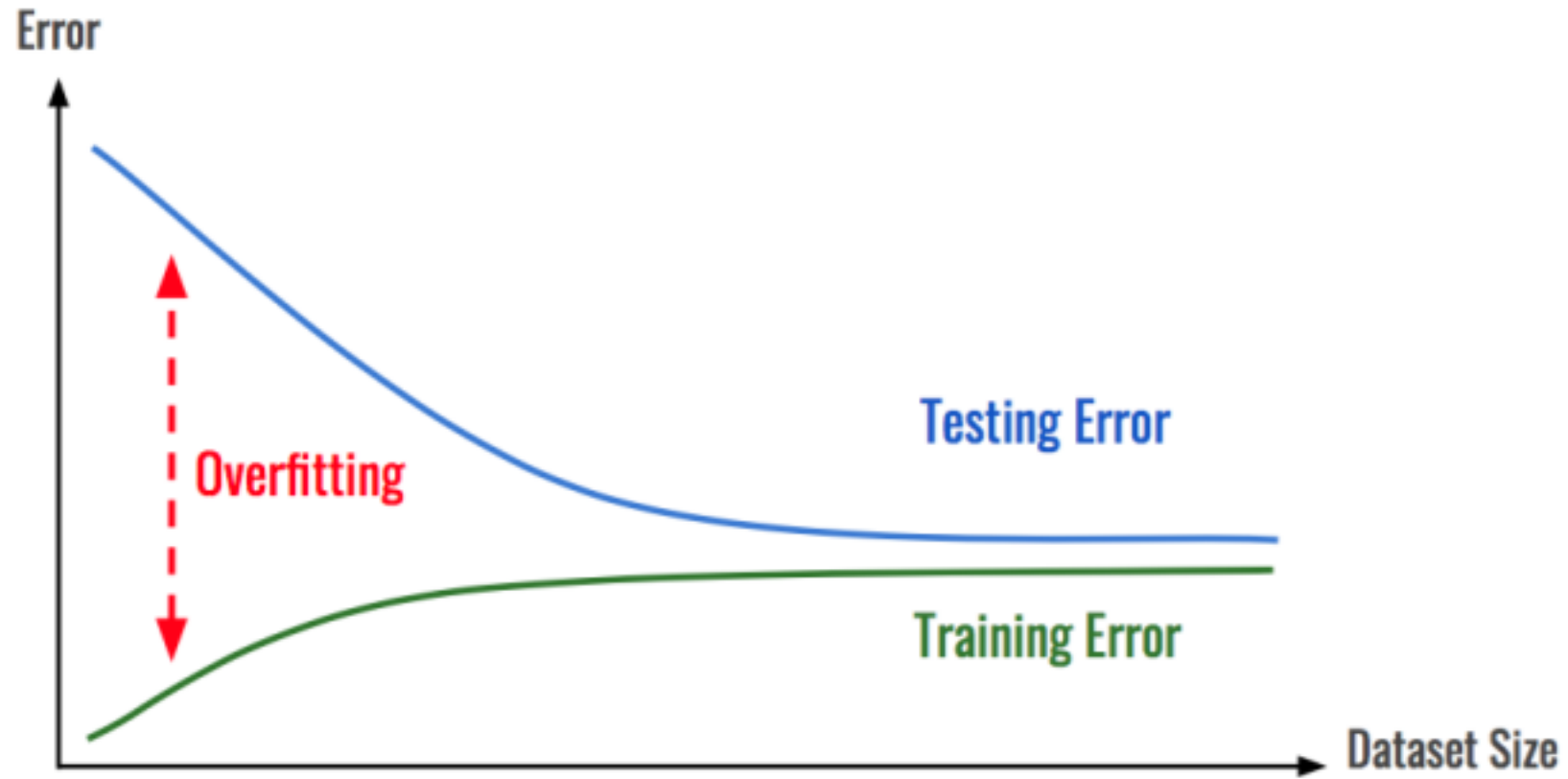
# Bias vs Variance

- ❑ High Bias
  - ❑ Problem with training data performance
  - ❑ Bigger network
  - ❑ More units in a layer
  - ❑ NN architecture search
- ❑ High variance
  - ❑ More data
  - ❑ Regularization
  - ❑ NN architecture search
- ❑ Bias variance tradeoff

# How to overcome overfitting?

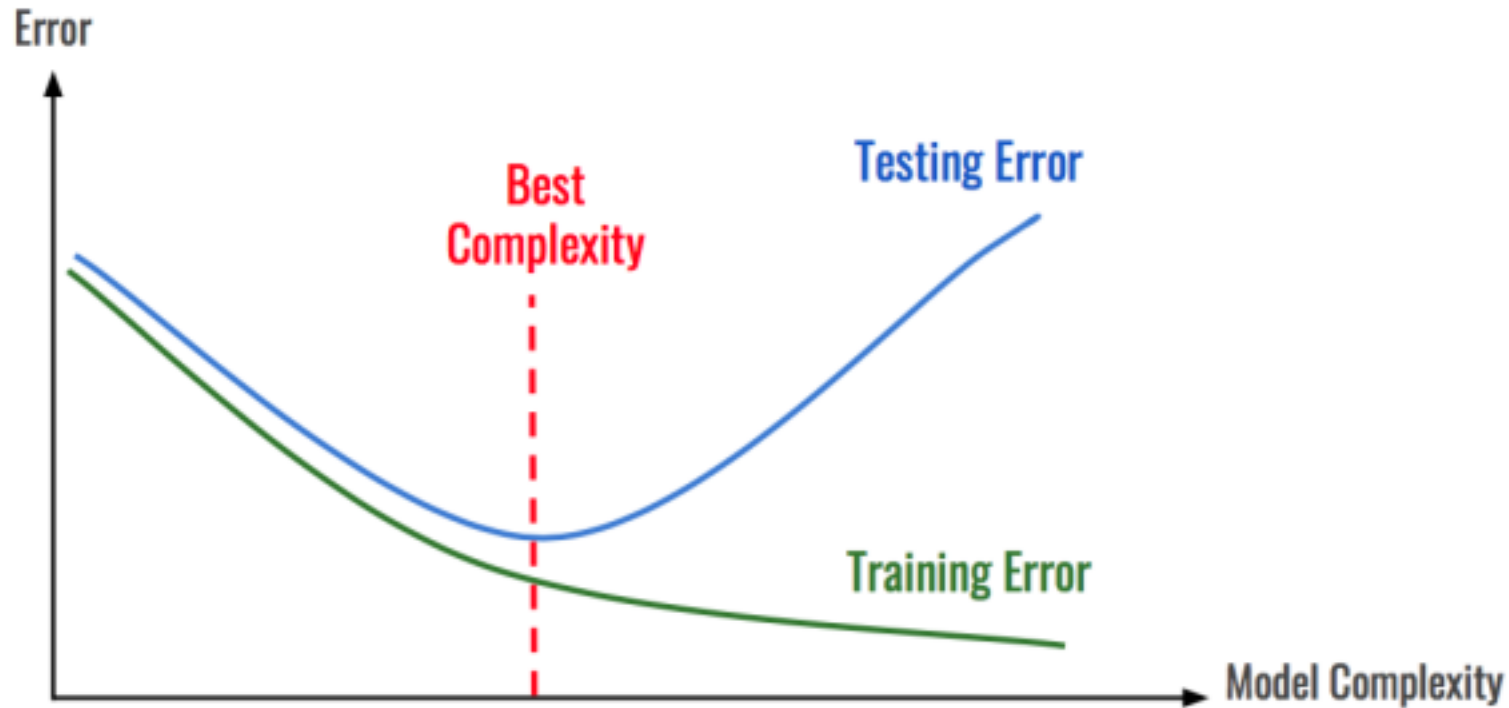
- ☐ Collect more Data
- ☐ Simplify the model
- ☐ Regularization
- ☐ Dropout Regularization
- ☐ Data Augmentation and noise
- ☐ Early stopping

# Get more Data



The **more data** you get, the **less** likely the model is to **overfit**.

# Simplify the Model



On the left, the model is too simple. On the right it overfits.

# Regularization

- Update the cost function
- $J(W, b) = \frac{1}{m} \sum_{i=1}^m \mathcal{L}(\hat{y}^i, y^i) + \frac{\lambda}{2m} ||W^2||_2$
- $J(W, b) = \frac{1}{m} \sum_{i=1}^m \mathcal{L}(\hat{y}^i, y^i) + \frac{\lambda}{m} ||W||_1$

# Regularization

## □ L1 and L2 regularization

One of the most powerful and well-known technique of regularization is to **add a penalty** to the **loss function**. The most common are called L1 and L2:

- The L1 penalty aims to minimize the absolute value of the weights
- The L2 penalty aims to minimize the squared magnitude of the weights.

$$\lambda \sum_{j=0}^M |W_j|$$

L1 Penalty

$$\lambda \sum_{j=0}^M W_j^2$$

L2 Penalty

# Regularization

- ❑ With the penalty, the model is forced to make compromises on its weights, as it can no longer make them arbitrarily large. This makes the model more general, which helps combat overfitting.
- ❑ The L1 penalty has the added advantage that it enforces feature selection, which means that it has a tendency to set to 0 the less useful parameters. This helps identify the most relevant features in a dataset.

Here is what the weight matrixes would look like. Note how the **L1** matrix is **sparse** with many zeros, and the **L2** matrix has *slightly smaller weights*.



Baseline



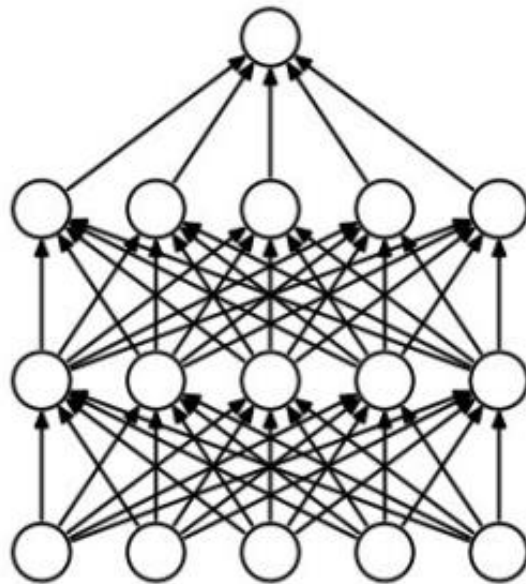
L1 Regularization



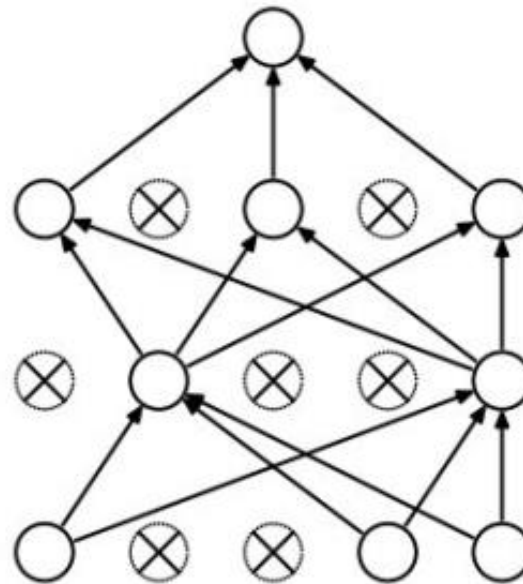
L2 Regularization

# Dropout and Dropconnect

This **extremely effective** technique is special **Deep Learning**, as it relies on the fact that neural networks process the information from one **layer** to the next. The idea is to randomly deactivate either **neurons (dropout)** or **connections (dropconnect)** during the training.



(a) Standard Neural Net



(b) After applying dropout.

# Data Augmentation and Noise

- ❑ one way of regularization
- ❑ Generate data using different transformation, scaling etc.



Each iteration sees as different variation of the original sample.

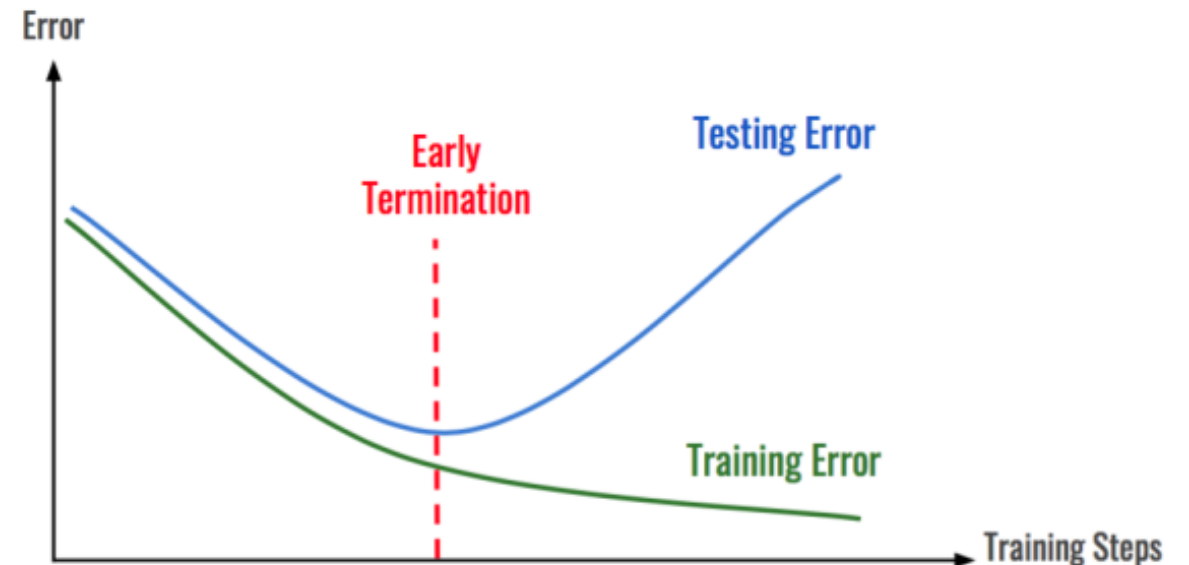
- ❑ Another good practice is to add Noise

- To the input: This serves the same purpose as data augmentation, but will also work toward making the model robust to natural perturbations it could encounter in the wild.
- To the output: Again, this will make the training more diversified.



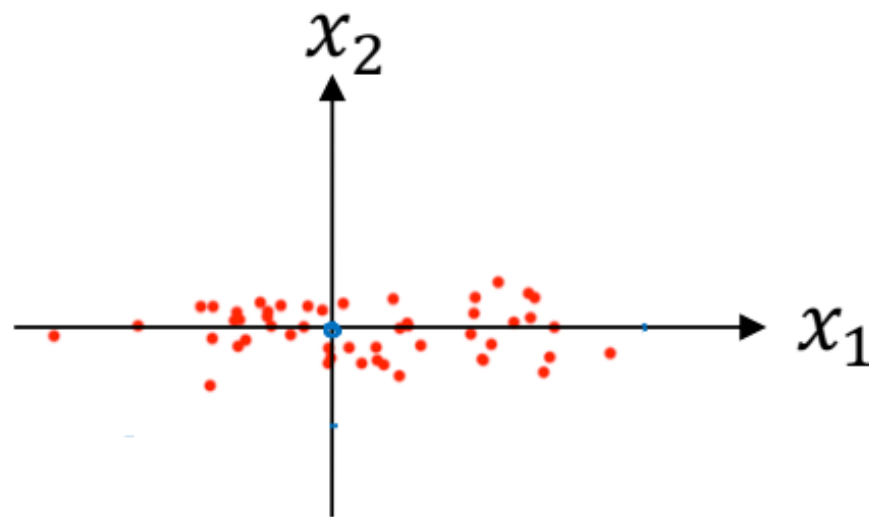
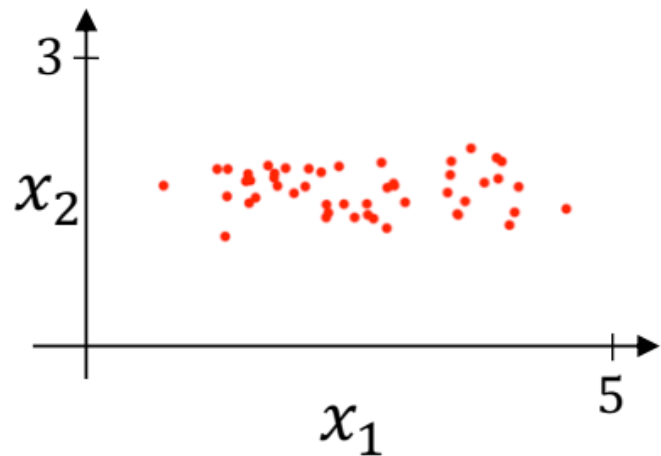
# Early Stopping

- ❑ In most cases, the model starts by learning a correct distribution of the data, and, at some point, starts to overfit the data. By identifying the moment where this shift occurs, you can stop the learning process before the overfitting happens.
- ❑ Stops early to avoid the overfitting
- ❑ Problem: orthogonalization



# Normalization

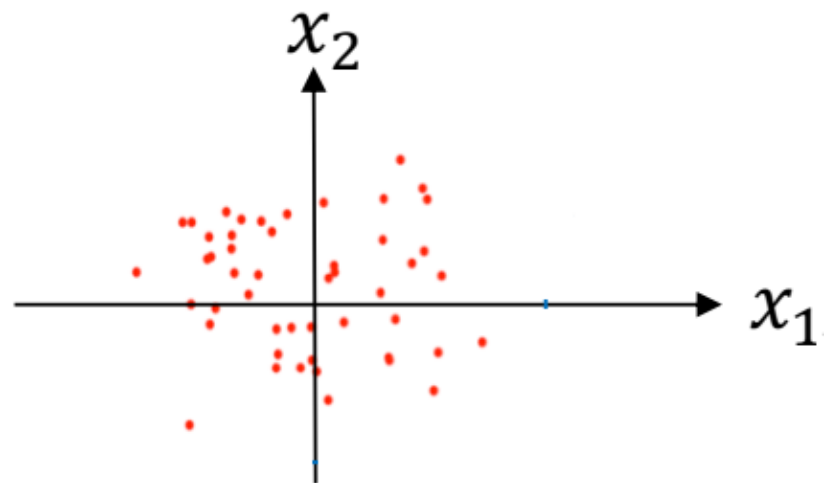
- ❑ Features can have different ranges (scale)
- ❑ This helps in efficient learning



Subtract mean: zero mean

$$\mu = \frac{1}{m} \sum_{i=1}^m x^i$$

$$x = x - \mu$$



Normalize variance

$$\sigma^2 = \frac{1}{m} \sum_{i=1}^m x^{i2}$$

$$x /= \sigma^2$$

# Mean normalization

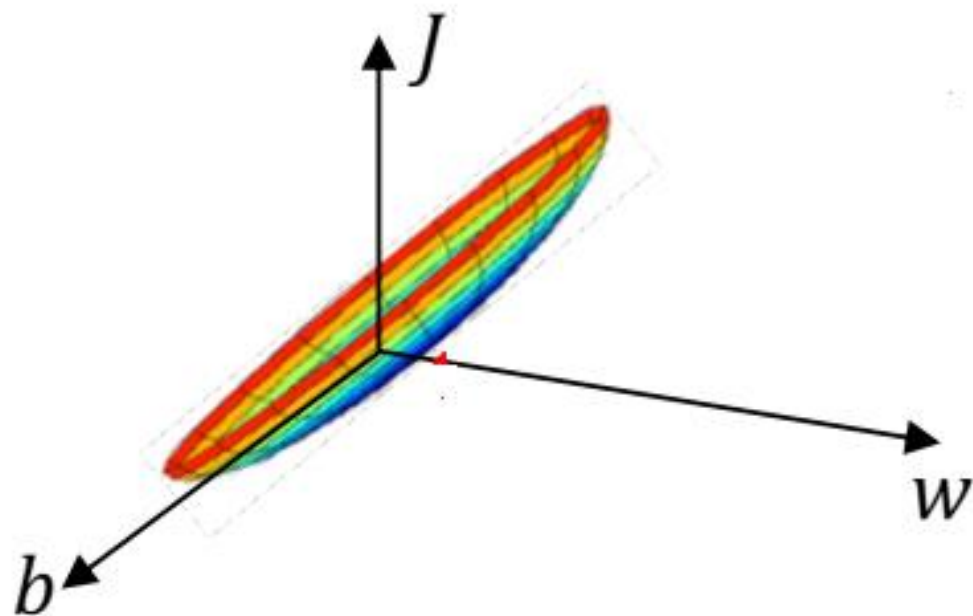
Replace  $x_i$  with  $x_i - \mu_i$  to make features have approximately zero mean  
(Do not apply to  $x_0 = 1$ ).

E.g.  $x_1 = \frac{size - 1000}{2000}$

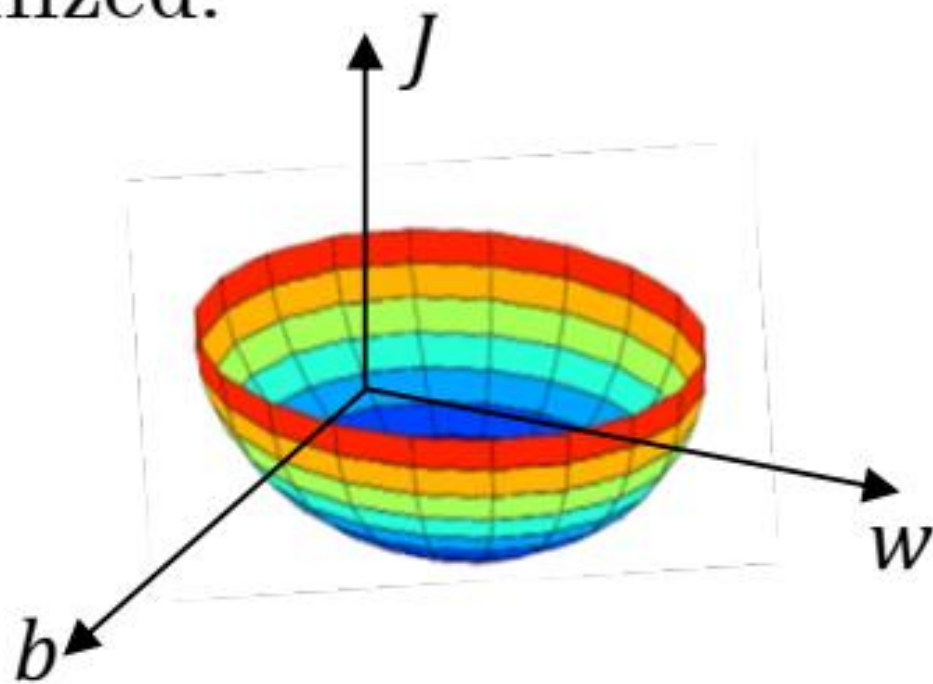
$$x_2 = \frac{\#bedrooms - 2}{5}$$

$$-0.5 \leq x_1 \leq 0.5, -0.5 \leq x_2 \leq 0.5$$

Unnormalized:



Normalized:

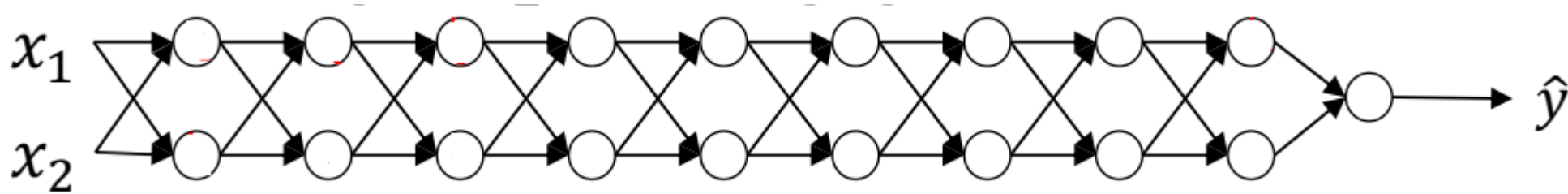


# Vanishing and Exploding Gradient

- ❑ Exploding Gradient

- ❑ Vanishing Gradient

# Example of Deep Network



Solution: weight initialization

Make sure that weights has variance  $1/n$